

Final Project – Multiplayer Game over LAN

Contents

Overview..... 2

 Tic-tac-toe..... 2

 Connect 4..... 3

 Mancala..... 3

Goals..... 4

Motivations 4

Network Topology 5

Configuration Details..... 6

Communication Flow..... 7

Tools and Technologies 8

Demonstration 9

Problems encountered:..... 11

Sources:..... 15

Overview

This project implements a multiplayer game featuring Mancala, Tic-tac-toe, and Connect 4, designed to play over a Local Area Network (LAN). Players connect through a client-server architecture, where the server manages game logic, including turn validation, win conditions, and synchronization, while the client provides an interactive user interface for players to make their moves.

The implementation uses Python, leveraging its socket programming capabilities to enable real-time communication between the server and clients.

Tic-tac-toe

The game is played on a 3x3 grid.
One player uses "X" and the other "O."

Gameplay:

Players take turns placing their marker in an empty square.

The objective is to align three markers in a row, column, or diagonal.

```
Starting Tic Tac Toe! You are 'X'.
Your move (row column: 0 0)(ranges: 0-2):

|   |   |
-----
|   |   |
-----
|   |   |
>
```

End of Game:

The game ends when a player aligns three markers (winning), or all nine squares are filled without a winner (draw).

Connect 4

```
Starting Connect 4! You are Player 1 (X).  
Your turn! Choose a column (0-6):  
| | | | | | | |  
| | | | | | | |  
| | | | | | | |  
| | | | | | | |  
| | | | | | | |  
| | | | | | | |  
0 1 2 3 4 5 6  
>
```

The game is played on a 7-column by 6-row grid.

Players alternate dropping tokens into a column, and the token occupies the lowest available space in that column.

Gameplay:

Players take turns dropping tokens into columns.

The objective is to connect four tokens in a row, column, or diagonal.

End of Game:

The game ends when a player connects four tokens (winning) or the grid is completely filled without a winner (draw).

Mancala

The board consists of two rows of six pits, each initially filled with an equal number of stones (typically 4).

Each player has a large storage pit, called a "Mancala," on their side of the board.

Gameplay:

Players take turns picking up all stones from one of their pits and distributing them counterclockwise, one by one, into subsequent pits, including their own Mancala but skipping the opponent's Mancala.

If the last stone lands in the player's Mancala, they get another turn.

If the last stone lands in an empty pit on the player's side, they capture all stones in the directly opposite pit on the opponent's side, along with the stone they placed.

End of Game:

The game ends when all six pits on one side of the board are empty.

The player with stones remaining in their pits adds them to their Mancala.

The winner is the player with the most stones in their Mancala.

```

=====
Player 2 (Pits 12-7):
  4 4 4 4 4 4
0                               0
  4 4 4 4 4 4
Player 1 (Pits 0-5):
=====

```

Goals

The project aims to showcase the integration of networking concepts into game development by creating a multiplayer experience. It focuses on enabling real-time interaction between two players over a LAN, ensuring seamless updates to the game board as moves are made. Through this project, the mechanics of game synchronization and real-time communication in a networked environment are demonstrated.

Motivations

I chose this project because I am fascinated by how multiplayer games enable real-time interactions between players, bridging physical distances to create shared experiences. By implementing a networked game, I aimed to explore the critical role of network communication in delivering engaging interactions. This project presented an opportunity to delve deeper into the technical challenges and design principles that power multiplayer environments, such as latency handling, synchronization, and data transmission.

Throughout the development process, there were moments when I felt stuck and doubted whether I would be able to complete the project. Debugging network issues or managing unforeseen obstacles often felt overwhelming. However, by staying persistent and methodically tackling each problem, I discovered new solutions and grew more confident in my skills. This process reinforced the importance of resilience and a problem-solving mindset.

While I acknowledge the server is not yet the most secure or fully optimized, I am proud of the progress I made and the knowledge I gained. Creating this project was a rewarding experience that deepened my understanding of network communication and game development.

Network Topology

This project uses a client-server model operating over a Local Area Network (LAN):

Communication between the client and server follows a simple request-response pattern. When it's a player's turn, the server sends a prompt to the client asking for a move. After each move, the server sends the updated game state to both players. The server and clients communicate over TCP sockets. Each player runs a separate client instance on their machine, and they connect to the same server to play a game.

Server (VM or physical computer):

- Acts as the central hub for game coordination.

- Listens for incoming client connections on a specific IP (HOST) and port (PORT).

- Handles client communication, game selection, and game execution.

Client (VM or physical computer):

- Connect to the server using its IP and port.

- Participate in a handshake to establish the connection.

- Receive instructions or game state updates from the server.

- Send player moves or responses back to the server.

Physical or Logical Layout:

- The server operates on a fixed IP (server's IP)

- Clients are dynamically connected devices on the same LAN or reachable network.

- No routers or switches are explicitly managed in the code.

Configuration Details

Server Configuration

IP Address:

The server is configured to bind to 0.0.0.0, making it accessible on all network interfaces of the host machine. This allows clients to connect to the server using the host's public, private, or localhost IP, depending on the network setup.

Port:

The server listens on port 65432. This port must be open and not blocked by a firewall for clients to connect. This applies when the devices are on a public network. The server also logs activities such as new connections, handshake success/failure, and errors to server.log. This aids in monitoring and debugging.

Maximum Connections:

The server accepts up to 2 clients concurrently (MAX_CONNECTIONS = 2), which matches the requirement for a two-player game.

Timeouts:

A 5-second timeout is set during the initial handshake process to ensure timely responses from clients. Once the handshake is successful, the timeout is removed for the remainder of the session.

Multithreading:

The server handles each client in a separate thread for efficient interaction. Without multithreading, the server would be blocked waiting for one client's actions (sending or receiving messages) and unable to handle other clients at the same time. Multithreading allows the server to manage the communication and gameplay logic for both clients concurrently. ¹

Client Configuration

Clients need to specify the server's IP address in SERVER_HOST before connecting.

The local IP was used to connect clients on LAN.

Port:

The client must connect to the same port (65432) on which the server is listening.

Handshake Protocol:

A custom handshake is implemented for connection validation:

Server sends: HANDSHAKE:HELLO

Client responds: HANDSHAKE:ACK

Server confirms: HANDSHAKE:SUCCESS

Failure to follow the protocol leads to disconnection.

¹ I still need more practice but I tried my best.

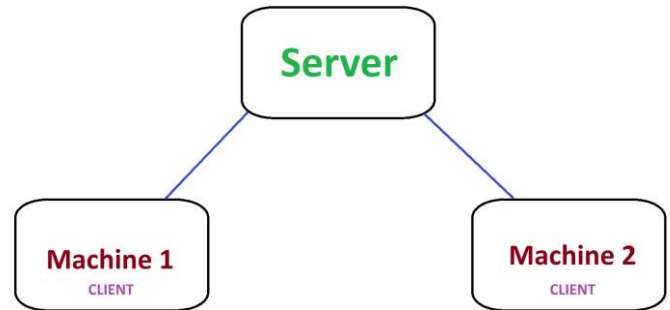
Communication Flow

Client Connection:

Each client establishes a TCP connection with the server. The server listens for incoming connections and accepts them.

The server waits for two clients to connect before proceeding with the game selection.

Upon successful connection, the server performs a handshake with each client to ensure valid communication (using the HANDSHAKE:HELLO and HANDSHAKE:ACK protocol).



Game Selection:

After the handshake, Player 1 (the first client to connect) is asked to choose a game (Tic-Tac-Toe, Connect 4, or Mancala).

Player 1 sends their choice to the server, which is then broadcast to Player 2 (the second client).

Player 2 waits for Player 1 to choose the game before responding with their own input or acknowledgement.

The server places the chosen game in a queue, ensuring the game is consistent across both clients.

Gameplay:

After the game is selected, the server runs the game logic (play_tic_tac_toe, play_connect4, or play_mancala) and controls the flow of the game.

The server listens for each player's input, ensuring that it alternates between Player 1 and Player 2.

Input from both players is validated by the server (ensuring the correct player makes a move and that the move is valid within the game's rules).

After each player's turn, the server processes the move and updates the game state.

The server broadcasts the updated state to both players to reflect the current status of the game.

Termination:

Once the game reaches an end condition (win, draw, or timeout), the server determines the outcome (who won, or if it's a draw).

The server broadcasts the final game state to both players, showing the result.

The server sends a message to each client indicating the game has ended, and then both client connections are closed.

Tools and Technologies

Python Language: used on server, client and game logic files.

Sockets (TCP/IP): Used for communication between the server and clients.

Game Logic: Mancala, tic-tac-toe and connect 4 game rules are implemented in Python, including board initialization, move validation, turn switching, and winner determination.

Platforms: Server tested on Windows and Linux.

Demonstration

The image displays a demonstration of a game server and client application. It consists of three main components:

- Code Editor (Left):** Shows the Python code for the game logic. The `client.py` file includes a `main()` function that sets up a socket connection to the server. The `game_logic.py` file contains the `play_game()` function, which handles game logic, including move validation and winner determination.
- Windows PowerShell (Top Right):** Shows the output of the game session. It displays the game board state, the message "Game over! Player 1 (X) wins!", and the server shutting down.
- VirtualBox (Bottom Right):** Shows a Linux terminal window running the game. It displays the game board state, the message "Game over! Player 1 (X) wins!", and the server shutting down.

```

Command Prompt
=====
Player 2 (Pits 12-7):
  0 10 1 2 1 1
11          21
  0 0 0 0 0 1
Player 1 (Pits 0-5):
=====
Waiting for Player 2 to make a move...

Current board:
=====
Player 2 (Pits 12-7):
  0 10 1 2 2 0
11          21
  0 0 0 0 0 1
Player 1 (Pits 0-5):
=====
Your turn, Player 1. Choose a pit:
>5
Move completed.
You get an extra turn!
Final board:
=====
Player 2 (Pits 12-7):
  0 0 0 0 0 0
26          22
  0 0 0 0 0 0
Player 1 (Pits 0-5):
=====
Game Over! Player 2 wins!
Disconnecting...
Closing connection.

Game made possible thanks to: https://www.youtube.com/watch?v=dQw4w9WgXcQ
Server is shutting down...

```

```

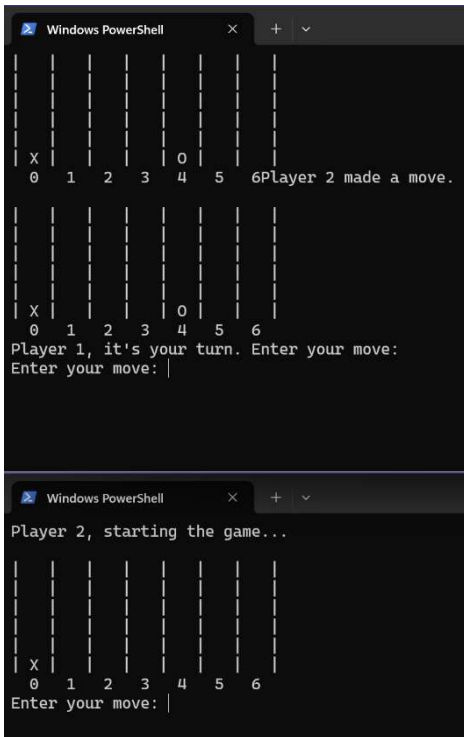
Ubuntu: ~/Documents
11          21
  0 0 0 0 0 1
Player 1 (Pits 0-5):
=====
Waiting for Player 1 to make a move...

Final board:
=====
Player 2 (Pits 12-7):
  0 0 0 0 0 0
26          22
  0 0 0 0 0 0
Player 1 (Pits 0-5):
=====
Game Over! Player 2 wins!
Disconnecting...
Closing connection.

Game made possible thanks to: https://www.youtube.com/watch?v=dQw4w9WgXcQ
Server is shutting down...
Client session closed.
@Ubuntu:~/Documents$

```

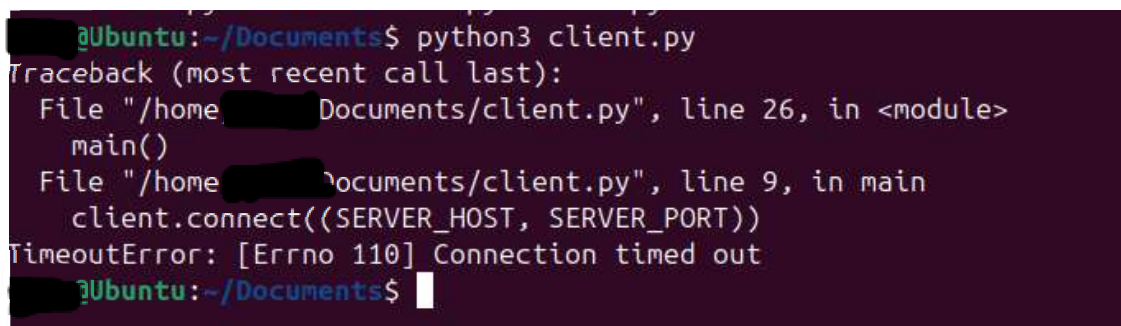
Problems encountered:



```
Windows PowerShell
| | | | | | | |
| X | | | | O | |
| 0 | 1 | 2 | 3 | 4 | 5 | 6 |
Player 2 made a move.
| | | | | | | |
| X | | | | O | |
| 0 | 1 | 2 | 3 | 4 | 5 | 6 |
Player 1, it's your turn. Enter your move:
Enter your move: |

Windows PowerShell
Player 2, starting the game...
| | | | | | | |
| X | | | | | | |
| 0 | 1 | 2 | 3 | 4 | 5 | 6 |
Enter your move: |
```

- Server would accept input from both players at the same time, regardless of whose turn it was.
- Game boards were not displayed correctly.



```
@Ubuntu:~/Documents$ python3 client.py
Traceback (most recent call last):
  File "/home/.../Documents/client.py", line 26, in <module>
    main()
  File "/home/.../Documents/client.py", line 9, in main
    client.connect((SERVER_HOST, SERVER_PORT))
TimeoutError: [Errno 110] Connection timed out
@Ubuntu:~/Documents$
```

- Couldn't connect to the server; Realized I entered the IP address incorrectly.

- I realized I didn't need to allow access to the port, but at least I learned how for future projects.

New Inbound Rule Wizard

Protocol and Ports

Specify the protocols and ports to which this rule applies.

Steps:

- Rule Type
- Protocol and Ports
- Action
- Profile
- Name

Does this rule apply to TCP or UDP?

☒ TCP
☐ UDP

Does this rule apply to all local ports or specific local ports?

☐ All local ports
☒ Specific local ports: 65432
Example: 80, 443, 5000-5010

New Inbound Rule Wizard

Name

Specify the name and description of this rule.

Steps:

- Rule Type
- Protocol and Ports
- Action
- Profile
- Name

Name:
gaming port

Description (optional):
port for project

```
PS C:\Users\ [redacted] \project> python -u "c:\Users\ [redacted] \project\game_logic\server.py"
Server started on 0.0.0.0:65432. waiting for players...
New connection from ([redacted]:13987)
New connection from ([redacted]:50570)

=====
Player 2 (Pits 12-7):
 4 4 4 4 4 4
0
 4 4 4 4 4 4
Player 1 (Pits 0-5):
=====

Player 1's turn.
Choose a pit (Player 1):
```

-The game board was displaying on the server side

The image is a collage of four terminal screenshots showing the execution of a Connect 4 game in Python. The first screenshot shows the game starting with Player 1 choosing 'Connect 4' and the board being initialized. The second screenshot shows Player 2 waiting for Player 1 to choose a column. The third screenshot shows the board after Player 1 has made a move. The fourth screenshot shows the board after Player 2 has made a move.

Terminal 1 (Top Left):

```

Welcome! Waiting for another player...
You are Player 1. Choose a game: (1) TicTacToe, (2) Connect 4, (3) Mancala
>2
Game chosen: 2
Starting Connect 4! You are Player 1 (X).
Your turn! Choose a column (0-6):
| | | | | | |
| | | | | | |
| | | | | | |
| | | | | | |
| | | | | | |
| | | | | | |
| | | | | | |
0 1 2 3 4 5 6
0
21
0 0

```

Terminal 2 (Top Right):

```

game made possible thanks to: https://www.youtube.com/w
PS C:\Users\Tere\Desktop\FALL 2024\CSCI 4345\project\ga
Welcome! Waiting for another player...

You are Player 2. Waiting for Player 1 to choose a game

Game chosen: 2

Starting Connect 4! You are Player 2 (O).
Waiting for Player 1 (X)...
| | | | | | |
| | | | | | |
| | | | | | |
| | | | | | |
| | | | | | |
| | | | | | |
| | | | | | |
0 1 2 3 4 5 6

```

Terminal 3 (Bottom Left):

```

| | | | | | |
| | | | | | |
| | | | | | |
| | | | | | |
| | | | | | |
| | | | | | |
| | | | | | |
0 1 2 3 4 5 6

C:\Users\Tere\Desktop\FALL 2024\CSCI 4345\project\ga
Welcome! Waiting for another player...

You are Player 2. Waiting for Player 1 to choose a game...

Game chosen: 2

Starting Connect 4! You are 'O'.
Waiting for the other player...
| | | | | | |
| | | | | | |
| | | | | | |
| | | | | | |
| | | | | | |
| | | | | | |
| | | | | | |
0 1 2 3 4 5 6

```

Terminal 4 (Bottom Right):

```

Command Prompt - python < X + v

C:\Users\Tere\Desktop\FALL 2024\CSCI 4345\project\ga
C:\Users\Tere\Desktop\FALL 2024\CSCI 4345\project\game_logi
Welcome! Waiting for another player...

You are Player 1. Choose a game: (1) TicTacToe, (2) Connect
>2
Game chosen: 2

Starting Connect 4! You are 'X'.

Choose a column (0-6):
| | | | | | |
| | | | | | |
| | | | | | |
| | | | | | |
| | | | | | |
| | | | | | |
| | | | | | |
0 1 2 3 4 5 6

```

Clients were not displaying game board and had input issues.

-Clients would not proceed after the first 3 steps (Connection-> player 1 chooses game-> player 1 makes a move).

```

Welcome! Waiting for another player...
You are Player 1. Choose a game: (1) TicTacToe, (2) Mancala, (3) Connect4
> 1
Game chosen: 1
Starting TicTacToe!Welcome to Tic Tac Toe!

  | |
--| |
  | |
--| |
  | |
Player X's turn.Enter row (0, 1, or 2):
0
0

```

The image displays two overlapping terminal windows from a Linux desktop environment. The top window, titled 'tere@Ubuntu:~/Documents', shows a Python script execution where Player X's turn was entered, followed by a traceback error: 'ConnectionResetError: [Errno 104] Connection reset by peer'. The bottom window, titled 'Command Prompt - python', shows a user prompt where the command 'python client.py' was executed, resulting in the message 'Welcome! Waiting for another player...'. Both windows have dark themes and standard terminal icons in their title bars.

```
tere@Ubuntu:~/Documents
| |
| |
Player X's turn.Enter row (0, 1, or 2):
0
Traceback (most recent call last):
File "/home/tere/Documents/client.py", line 26, in <module>
    main()
File "/home/tere/Documents/client.py", line 13, in main
    server_message = client.recv(1024).decode()
ConnectionResetError: [Errno 104] Connection reset by peer
tere@Ubuntu:~/Documents$ 
command not found
tere@Ubuntu:~/Documents$ 
command not found
tere@Ubuntu:~/Documents$ python3 client.py
Welcome! Waiting for another player...

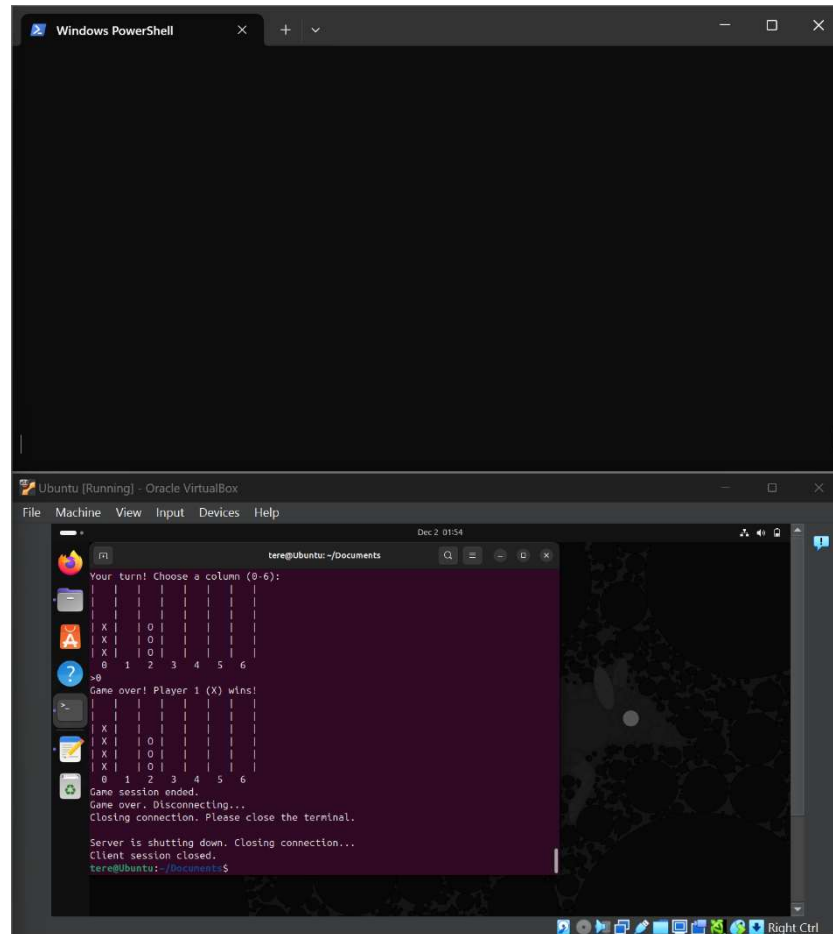
You are Player 2. Waiting for Player 1 to choose a game...
```

```
Command Prompt - python C:\Users\SC1 4345\project\game_logic\python client.py
Welcome! Waiting for another player...

>
```

- Clients were not synchronized

-Terminals would not close properly



Sources:

[socket programming](#)

[more socket programming](#)

[tutorialspoint - python socket programming](#)

[python.org](#)

[mancala: how to play](#)